Overview

The Qualcomm Image Loader (QIL) is a tool for flashing firmware images to Qualcomm-based devices. It supports Windows, Linux and WSL.

Open Source Repository

https://github.com/qualcomm/qualcomm-image-loader

Distributed Components

QIL is a zipped package with list of below component bundled:

- QIL executable
- QIL User Guide
- Qualcomm User Space Driver (install optional)

Supported Operating Systems

OS	Description
Linux x64	Ubuntu 24.04 and newer
Windows x64	Windows 11
Windows ARM64	Windows 11
WSL	Ubuntu 24.04 and newer

Setup

1. Download the zip package for your platform from Qualcomm Software Center (QSC) – Search for “Qualcomm Image Loader”
2. Unzip the downloaded zip package.
3. Driver installation(optional)- Installing Qualcomm user-space drivers is optional if the Qualcomm kernel drivers are already present. If drivers have not been installed, the user-space driver included in this package may be used as an alternative. Driver installation is a one-time process; please refer to the "README" provided with the accompanying user-space driver zip bundle for detailed installation instructions.

Usage

Prerequisites

Before using QIL, ensure the device is in EDL mode.

Steps to Put the Device into EDL Mode for build loading

There are two methods on how to set a device to edl mode:

1. Use the device hardware button to put the device in EDL mode, if applicable. Refer to your device documentation for details.

   Some devices feature a hardware switch for EDL mode. Common methods:

   - Hold both volume buttons while powering on.
   - Press and hold the F_DL button while powering on.

2. Set the device to EDL via ADB.

   Ensure ADB is installed. Refer to the official Android Debug Bridge (ADB) documentation online for installation instructions.

   Run the following commands to ensure the device is in a good state:

   ```
   adb root

   adb wait-for-device

   adb shell mount -o remount, rw /

   adb wait-for-device
   ```

   Run one of the following command to set the device into EDL mode:

   ```
   adb shell reboot edl
   ```

   or

   ```
   adb reboot edl
   ```

To verify that the device has successfully entered Emergency Download (EDL) mode, run the following command:

```
./qil --devices
```

EDL Mode in Linux Verification

If the device is in EDL mode, the output will include a line similar to:

```
Protocol Type: 3 --- /dev/QCOM_LIBUSB_QDLoader_9008_2-2:1.0 EDL device found!!!
```

(See the screenshot below for an example.)

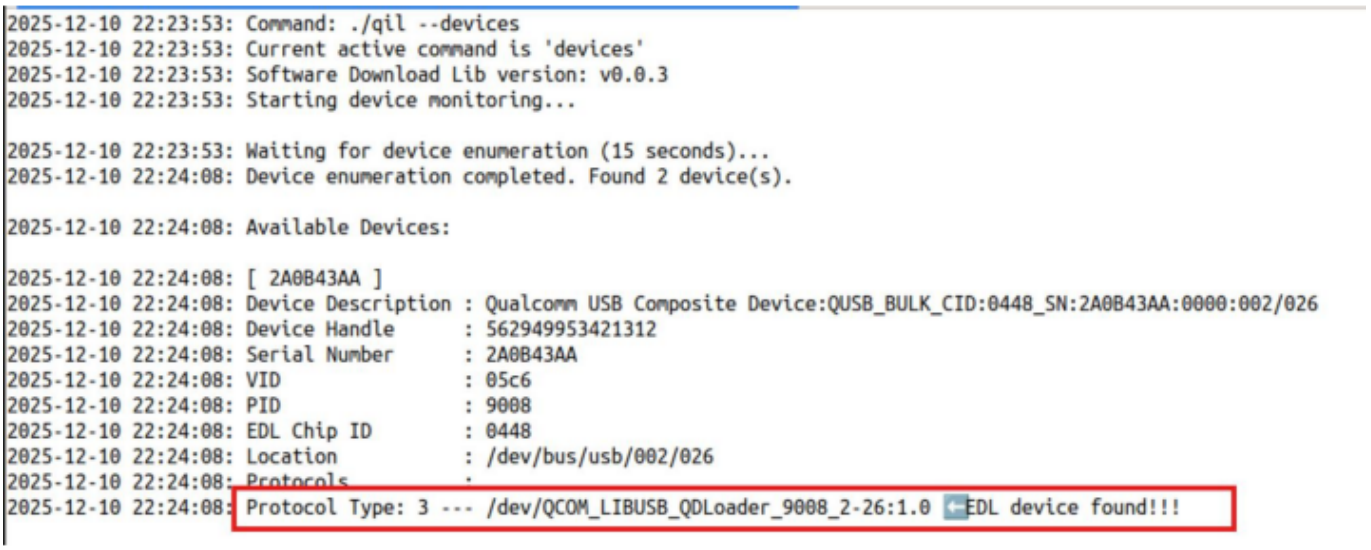

Figure 1 - Successfully set device to EDL mode (Linux)

EDL Mode in Windows Verification

If the device is in EDL mode, the output will include a line similar to:

```
Protocol Type: SAHARA --- Qualcomm HS-USB QDLoader 9008 EDL device found!!!
```

(See the screenshot below for an example.)

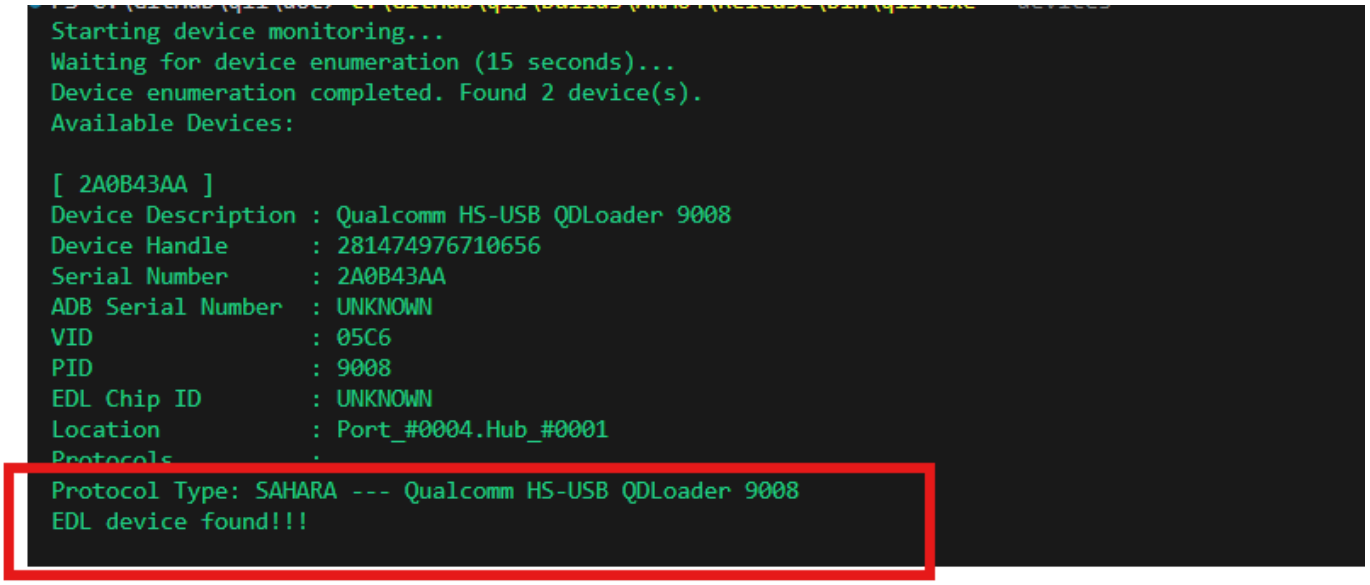

Figure 2 - Successfully set device to EDL mode (Windows)

EDL Mode in WSL Verification

If the device is in EDL mode, the output will include a line similar to:

```
Protocol Type: SAHARA --- Qualcomm HS-USB QDLoader 9008 EDL device found!!!
```

(See the screenshot below for an example.)

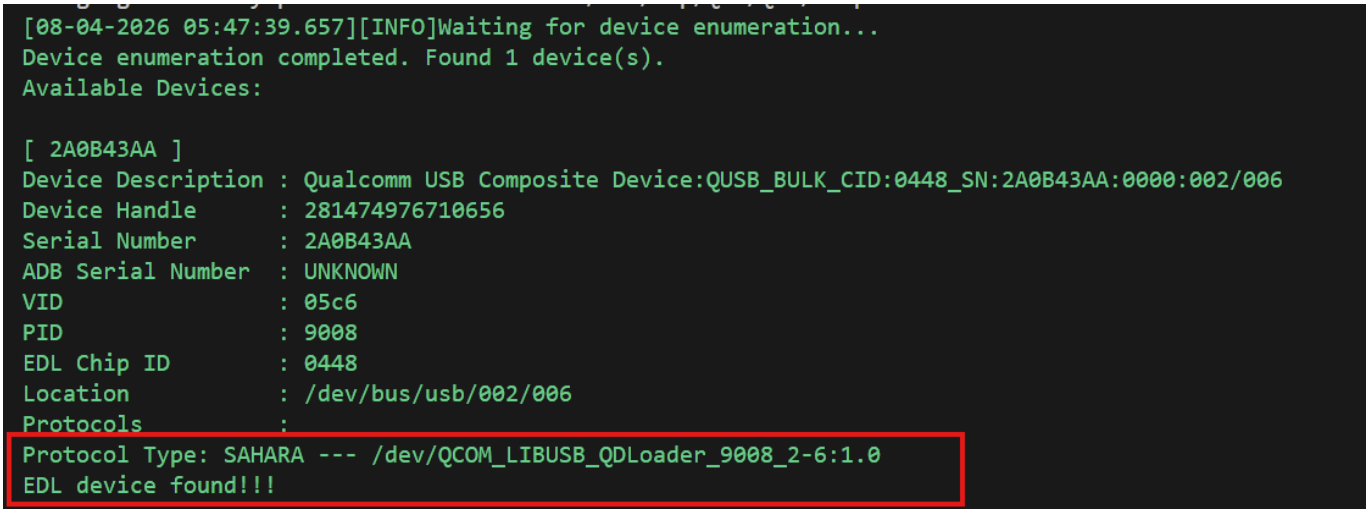

Figure 2 - Successfully set device to EDL mode (Windows)

Run QIL

Flash a flat build image to a device:

./qil --device= --flash-build --reset=true --build=/path/to/flat/build --memory-type=UFS

Note: If you are using WSL as the host, please add `--zlp-aware-host=false` to ensure proper USB communication, such as:

```
./qil --device=<MSM SN> --flash-build --reset=true --build=/path/to/flat/build --memory-type=UFS --zlp-aware-host=false
```

For a comprehensive list, refer to QIL Commands.

Supported Features

QIL Commands

To view a list of all available commands, run ./qil --help or ./qil -h from the command line. The following table lists all supported QIL commands, along with a brief description of each command and its required and optional arguments.

See QIL Arguments for more information about the arguments.

The following table lists all available arguments, their expected values, and descriptions.

Command	Required Arguments	Optional Arguments	Command Description
Flash Operations			

`--flash-build`	`--build`
	`--device`
	`--memory-type`
	`--reset`
		`--read-image-path`
		`--slot`
		`--erase`
		`--device-programmer`
		`--cdt`
		`--active-partition`
		`--chained-digest`
		`--signed-digest`
		`--validation-mode`
		`--raw-program`
		`--partition-index`
		`--patch-program`
		`--preserve-partitions`
		`--skip-sahara`
		`--firehose-init-time`
		`--firehose-rx-timeout`
		`--validate-image-size`
		`--port-trace`
		`--zlp-aware-host`
		`--verbose`
			Flash firmware build to device.

`--send-xml`	`--device`
	`--device-programmer`
	`--memory-type`
	`--reset`
	`--xml-path`
		`--slot`
		`--skip-sahara`
		`--firehose-init-time`
		`--firehose-rx-timeout`
		`--port-trace`
		`--zlp-aware-host`
		`--verbose`
			Send a firehose command sequence in an XML file. Can be used to send peek command.

`--send-image`	`--device`
	`--device-programmer`
	`--memory-type`
	`--image-path`
	`--lun`
	`--reset`
	`--start-sector`
		`--slot`
		`--skip-sahara`
		`--firehose-init-time`
		`--firehose-rx-timeout`
		`--port-trace`
		`--zlp-aware-host`
		`--verbose`
			Send a binary image to a user defined region in device (with partition index and start index).

UFS Provisioning			

`--ufs-provision`	`--device`
	`--device-programmer`
	`--ufs-provision-xml`
		`--slot`
		`--chained-digest`
		`--signed-digest`
		`--skip-sahara`
		`--firehose-init-time`
		`--firehose-rx-timeout`
			Execute a UFS provision.

		--port-trace --zlp-aware-host --verbose	
Create Digest Files			
--create-flash-build-vip-digest	--build --memory-type --out --reset	--slot --erase --cdt --validation-mode --raw-program --patch-program --digest-header-type --port-trace --zlp-aware-host --verbose	Create flash build VIP digest. Offline process, no device needed.
--create-ufs-provision-vip-digest	--out --ufs-provision-xml	--slot --digest-header-type --port-trace --zlp-aware-host --verbose	Create UFS provision VIP digest. Offline process, no device needed.
--create-validation-digest	--build --memory-type --out	--raw-program --port-trace --zlp-aware-host --verbose	Create build validation digest. Offline process, no device needed.
Erase Device Partitions			
--erase-partitions	--device --device-programmer --memory-type	--slot --partition-index --skip-sahara --firehose-init-time --firehose-rx-timeout --port-trace --zlp-aware-host --verbose	Erase specified partitions in device.
Read from Device			
--get-flash-info	--device --device-programmer --memory-type --reset	--slot --partition-index --skip-lun-info --skip-sahara --firehose-init-time --firehose-rx-timeout --port-trace --zlp-aware-host --verbose	Get device flash information only.

`--read-images`	`--build`
	`--device`
	`--memory-type`
	`--read-image-path`
	`--reset`
		`--slot`
		`--erase`
		`--device-programmer`
		`--raw-program`
		`--skip-sahara`
		`--firehose-init-time`
		`--firehose-rx-timeout`
		`--port-trace`
		`--zlp-aware-host`
		`--verbose`
			Read partition images from device.

Query Devices			
`--devices`		`--verbose`	List all available device identifiers.
Device Control & Utilities			
`--reset-device`	`--device`	`--port-trace` `--verbose`	Reset device from firehose mode. Normally used when the previous command was executed with `--reset=false`.
`--help`, `-h`			Display help information.
`--version`			Display QIL application version.

QIL Arguments

Option	Value	Description
`--active-partition`	`<0|1|...>`	Set the bootable partition index during flat build download. (e.g., `1` for UFS and `0` for eMMC).
`--build`	`<FLAT_BUILD_PATH>`	Absolute path to flat build directory containing: • **Flash images**: Search path for programmer, download binary, or XML • **Create VIP digest**: Search path for download binary or XML • **Create validation digest**: Search path for download binary or XML
`--cdt`	`<CDT_PATH>`	Given absolute path to CDT (Configuration Data Table) binary file. Used to replace CDT binary during --flash-build.

`--chained-digest`	`<DIGEST_PATH>`	Given absolute path or relative path to --build path for chained digest file, used for VIP download. Must be used together with signed digest file. Chained digest file AND signed digest file needs to be created before performing a VIP download. See Digest file creation section.
`--device`	`<ID>`	Specify target device identifier for device operation. Use `SERIAL NUMBER` or `ADB Serial Number` or `Device Description` from `--devices` command.
`--device-programmer`	`<PROGRAMMER_PATH>`	Given absolute path to a device programmer file or sahara XML used during Sahara transfer. Can be relative path if --build path is also given in some process.
`--digest-header-type`	`<ELF|MBN>`	Configure digest header type for VIP digest only used in create VIP digest. If --digest-header-type is not set, will create MBN type digest.
`--erase`		Specify enabling erasure of a specific partition before download. If --partition-index is not specified, it will erase default configuration (0,1,2,4,5 for UFS and 0 for eMMC).
`--firehose-init-time`	`<TIME_IN_MS>`	Configure firehose initialization time in ms. Increase for slower devices or connections.
`--firehose-rx-timeout`	`<TIME_IN_MS>`	Configure firehose data reception timeout in ms. Increase for slower devices or connections.
`--image-path`	`<IMAGE_PATH>`	Specify binary image path for --send-image.
`--partition-index`	`<PARTITION_INDEXES>`	Comma-separated list of partition indexes to erase during download, erase partition only or get flash info. • Flash Images: Only erase specified partition, but this does not impact the download partition which is configured by rawprogram • Get flash info: Specify the partition index to get information on • Erase flash only: Specify the flash partition to be erased.

`--patch-program`	`<PATCH_XML_LIST>`	Semicolon-separated list of patch XML files. Specify dedicated XML files to override auto-detection of patch files, only used in download build.
`--lun`	`<LUN_NUMBER>`	Configure partition index for --send-image.
`--memory-type`	`<UFS|EMMC|NAND|SPINOR>`	Specify type of flash memory on the target device.
`--port-trace`		Enable port trace logging. Writes raw TX/RX data to port-trace files in the log directory for protocol-level debugging.
`--raw-program`	`<RAW_XML_LIST>`	Semicolon-separated list of rawprogram XML files. Specify dedicated XML file if you don't want to use default list, only used in download build & read images.
`--read-image-path`	`<DIR_PATH_FOR_READ>`	Directory path to store read-out binary images: • Flash build: Stores binaries read back from device during validation (validation-mode 1 or 3 only) • Read images: Output directory for partition images read from device
`--reset`	`<true|false>`	Enable or disable reset device after firehose process completion. Device will remain in firehose mode if set to false.
`--signed-digest`	`<DIGEST_PATH>`	Given absolute path or relative path to --build path for signed digest file, used for VIP download. Signed digest file needs to be created before performing a VIP download. See Digest file creation section.
`--skip-lun-info`		Enable skipping getting detailed LUN info for --get-flash-info.
`--skip-sahara`		While device already in firehose mode. Enable skipping the transfer of device programmer using Sahara. Normally used in a subsequent command when the previous command was executed with '--reset=false' or when the previous command failed and the device remains in firehose mode.
`--slot`	`<SLOT_INDEX>`	Configure Memory slot number. Default to slot 0 if not specified.

`--start-sector`	`<START_SECTOR_NUMBER>`	Configure start sector for --send-image.
`--out`	`<DIR_PATH_FOR_READ>`	Output path to save generated digest file. Must be used inside create VIP digest or build validation digest process.
`--ufs-provision-xml`	`<PROVISION_XML>`	Given absolute path to UFS provision XML file. Only used together with --ufs-provision & --create-ufs-provision-vip-digest.
`--validate-image-size`		Validate image sizes against rawprogram.xml during download. Compares image file sizes with sizes specified in rawprogram.xml and fails if any image exceeds the defined size.
`--validation-mode`	`<0|1|2|3|4>`	Validate firmware images during download. No validation if not specified: • 0 - No validation • 1 - Binary Readback: Read back binary data from mobile -> create digest 1 from those flash data -> read original download binary file -> create digest 2 for download file in runtime -> compare digest 2 with digest 1 • 2 - SHA256 Readback: Read digest 1 from mobile directly -> read download binary file -> create digest 2 for original download file in runtime -> compare digest 2 with digest 1 • 3 - Binary readback with digest file (Requires Build Validation File): Read back binary data from mobile -> create digest 1 from those flash data -> load pre-created digest 2 from file -> compare digest 2 with digest 1 • 4 - SHA256 Readback with digest file (Requires Build Validation File): Read digest 1 from mobile directly -> load pre-created digest 2 from file -> compare digest 2 with digest 1
`--verbose`		Enable verbose logging output. Shows detailed operation logs and debug information.
`--xml-path`	`<XML_PATH>`	Configure command XML file path used by --send-xml. Can be used to send peek command.

`--zlp-aware-host`	`<true|false>`	Enable or disable ZLP (Zero Length Packet) aware host for USB transfers. **Important:** If you are using WSL (Windows Subsystem for Linux) as the host, please set this to `false` (i.e., `--zlp-aware-host=false`) to ensure proper USB communication.

Logs

Debug logs capture detailed information about QIL operations and can assist in troubleshooting. They can be found in the following location:

Linux

```
/var/tmp/QFS/QIL/Logs/
```

Windows

```
C:\ProgramData\QFS\QIL\Logs\
```

WSL

```
/var/tmp/QFS/QIL/Logs/
```

Limitations

1. **EDL Mode Required**

   All QIL device operations require the device to be in EDL mode. If the device is not in EDL mode, commands such as `./qil --devices` will not detect it. See the EDL mode section for setup instructions.

2. **Single Device at a Time**

   QIL supports flashing only one device at a time. Multi-device parallel operations are not supported.

FAQ

Below are some common issues and their solutions.

1. QIL reports that the device has not been found.

   Please ensure Qualcomm Drivers have been installed. See the Setup section for details.

   If drivers are installed and the device is still not detected, try stopping ADB. While QIL can run alongside ADB in most cases, stopping the ADB server may help resolve detection issues:

   ```
   adb kill-server
   ```

2. QIL reports that the device is not in EDL mode.

   Please ensure that the device has been set to EDL mode. See EDL mode section for details.